

DIAG NÓSTICO TÉCNICO & WIKI ARQUITETURAL

Avaliação de Maturidade, Transição de Senioridade e Plano de Ação para Produção em Rust

Data: Julho, 2026

Projeto: Core ETL / Async Pipeline

Autor: Engenharia / Arquitetura

1. Sumário Executivo & Diagnóstico Arquitetural

Esta documentação serve como diretriz técnica e registro oficial no repositório sobre a avaliação da infraestrutura assíncrona desenvolvida em Rust. O projeto apresenta um contraste marcado entre a **maturidade de design de software** (resultado de uma bagagem sólida em arquitetura) e a **curva de aprendizado das idiomáticas de runtime** da linguagem.

✓ PONTOS FORTES DE DESIGN

- Arquitetura Hexagonal Estrita:** Isolamento limpo entre as camadas `core`, `infra` e `shared`.
- Operadores Compostos (`FnOperator`):** Ciclo de vida desacoplado para pipelines de processamento ETL.
- Contexto Tipado:** Estado compartilhado sem acoplamento cego via `Arc<Mutex<T>>`.
- Zero Dívida Estrutural:** Não há acoplamento que exija refatoração destrutiva ou remodelagem de módulos.

⚠ APRENDIZADOS DE RUNTIME (RUST)

- Gargalo de Stream:** Uso indevido de `filter_map` ocasionando encerramento precoce de fluxos.
- Semântica de Broadcast:** Lag e descarte descontrolado de mensagens sob alta carga concorrente.
- Custo de Memória:** Uso excessivo de `Vec<u8>` em vez de buffer zero-copy com `Bytes`.
- Erosão de Contexto de Erro:** Colapso de Enums para `String` no tratamento de falhas.

Alerta de Disciplina de Engenharia: O maior risco do projeto no estágio atual não é técnico, mas de direcionamento. A elegância da base arquitetural pode induzir ao refinamento precoce de abstrações em `shared/utils` antes de resolver requisitos não-funcionais críticos e falhas reais de I/O na camada de infraestrutura.

2. Matriz de Requisitos Não-Funcionais (RNFs)

Para atingir o estado de prontidão para produção (*Production Readiness*), o sistema deve satisfazer rigorosamente os seguintes critérios de qualidade não-funcionais:

REQUISITO	CATEGORIA	META & CRITÉRIO DE ACEITE
RNF-01: Correctness em Streams	Concorrência / Reatividade	Garantir que nenhum combinator (ex: <code>filter_map</code>) feche o stream prematuramente ao receber elementos nulos ou erros recuperáveis.
RNF-02: Bounded Backpressure	Resiliência / Memória	Canais <code>broadcast</code> devem possuir limites de capacidade definidos e políticas explícitas de descarte ou desaceleração em cenários de lag.
RNF-03: Zero-Copy Efficiency	Performance / Latência	Fronteiras de I/O do ETL devem utilizar <code>bytes::Bytes</code> para evitar cópias desnecessárias de heap associadas a <code>Vec<u8></code> .
RNF-04: Error Context Preservation	Observabilidade / Debug	Preservar a árvore hierárquica de erros (causa raiz e stack trace) utilizando enums tipados com <code>thiserror</code> sem conversão para <code>String</code> .
RNF-05: Real I/O Persistence	Integridade de Dados	Substituir quaisquer retornos mockados ou caminhos fictícios por operabilidades reais de leitura/escrita em disco ou banco de dados.

3. Plano de Ação e Matriz de Escopo

A priorização a seguir rege as próximas atividades do repositório. O avanço em novas abstrações fica **congelado** até que os itens de prioridade máxima (P0 e P1) estejam completamente resolvidos e cobertos por testes automáticos.

PRIORIDADE	DOMÍNIO	PROBLEMA DETECTADO	AÇÃO ENGENHEIRADA
P0 - BLOQUEADOR	Infraestrutura / IO	Persistência com caminhos fictícios e retornos mockados.	Implementar a persistência real de dados no banco/disco e configurar tratamento de falha de gravação física.
P0 - BLOQUEADOR	Async Streams	Stream sendo finalizada precocemente no <code>filter_map</code> (#1).	Substituir combinators por tratamentos explícitos de <code>Option/Result</code> usando <code>then</code> ou <code>flat_map</code> .
P1 - CRÍTICO	Sync & Channels	Lag de leitores derrubando/perder dados no <code>broadcast</code> (#2).	Implementar buffer delimitado com estratégia de backpressure ou filas persistentes intermediárias.
P1 - CRÍTICO	Observabilidade	Erros sendo colapsados para <code>String</code> (#6).	Refatorar erros do sistema para Enums estruturados usando <code>thiserror</code> , eliminando <code>.map_err(e e.to_string())</code> .
P2 - PERFORMANCE	Gestão de Memória	Overhead de alocação recorrente em <code>Vec<u8></code> (#7).	Migrar payloads do pipeline para <code>bytes::Bytes</code> garantindo compartilhamento por referência sem cópia.
CONGELADO	Shared Abstractions	Refinamento de novos operadores em <code>shared/utils</code> .	Pausado. Bloqueado contra novas features até a conclusão dos testes de integração de P0/P1.

4. Boas Práticas Arquiteturais para o Wiki

1. Padrão de Tratamento de Erros Sem Perda de Contexto

É proibido converter erros internos em `String` na camada de infraestrutura ou core. Todos os erros devem ser propagados via Enums tipados:

```
#[derive(thiserror::Error, Debug)]
pub enum PipelineError {
    #[error("Falha na persistência I/O: {0}")]
    Io(#[from] std::io::Error),
    #[error("Erro no processamento da stream: {0}")]
    StreamLag(#[from] tokio::sync::broadcast::error::RecvError),
}
```

2. Diretrizes de Manipulação de Streams em Tokio

Nunca utilize combinators que tratem `None` como sinal de fim de fluxo quando a intenção for apenas ignorar um item inválido. Prefira o padrão explícito:

```
stream.filter(|item| async move { item.is_valid() })  
    .then(|item| async move { process(item).await })
```

3. Gestão de Buffers e I/O Contínuo

Para transmissões de dados em pipelines, prefira `bytes::Bytes` que utiliza contagem de referências internamente e evita desalocação/realocação de arrays na heap a cada operador do pipeline ETL.